



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

DESARROLLO DE UN GENERADOR DE CONTENIDO PARA JUEGOS CON RECURSOS LIMITADOS

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Antonio Luis Farrés Corzo

Tutor: Dr. C. Omar Correa Madrigal

La Habana, Junio de 2026

Año del Centenario del Comandante en Jefe Fidel Castro Ruz

La creatividad es la inteligencia divirtiéndose.
Albert Einstein

Dedicatoria

A mi familia, por ser el pilar fundamental en cada paso de mi formación profesional. A todos aquellos que, con su apoyo incondicional, hicieron posible la culminación de esta etapa académica.

Agradecimientos

A mi tutor, Omar Correa, por su guía, paciencia y dedicación a lo largo de todo este proceso investigativo. A la Facultad de Tecnologías Interactivas por brindarme las herramientas y el conocimiento necesarios para mi desarrollo profesional. A mis compañeros y amigos, por los intercambios de ideas y el apoyo mutuo durante estos años de formación. A todos los usuarios del sector creativo que participaron en las pruebas de validación y contribuyeron con sus perspectivas al perfeccionamiento de Assets Studio.

Declaración de autoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Antonio Luis Farrés Corzo

Autor

Dr. C. Omar Correa Madrigal

Tutor

La investigación aborda la ineficiencia en la gestión de activos digitales en equipos creativos que operan con recursos tecnológicos limitados. El objetivo general consiste en desarrollar y validar una aplicación web, denominada Assets Studio, orientada a organizar, recuperar y reutilizar recursos multimedia de forma eficiente en el contexto cubano. El diseño metodológico integra Design Thinking para la identificación de necesidades y el prototipado centrado en el usuario, junto con Scrum como marco de desarrollo iterativo para la implementación técnica. El estudio incluye entrevistas semiestructuradas, modelado de requisitos, construcción incremental del producto y pruebas de usabilidad con usuarios del sector creativo. Como resultado, se implementa una solución con autenticación, carga y categorización de recursos, búsqueda semántica, generación asistida por inteligencia artificial e interacción colaborativa. La validación evidencia una disminución del tiempo de localización de activos y una mejora en la organización del flujo de trabajo. Se concluye que la propuesta es técnicamente viable y metodológicamente pertinente para contribuir a la informatización de procesos creativos en escenarios de conectividad y recursos restringidos.

Palabras clave: Gestión de activos digitales, desarrollo de software, Design Thinking, Scrum, búsqueda semántica.

Lista de tareas pendientes	xiii
Introducción	1
1 FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO	3
1.1 Referentes teórico-metodológicos sobre el objeto de estudio	3
1.2 Fundamentos teórico-metodológicos del desarrollo tecnológico	4
1.3 Diagnóstico del objeto de estudio: aplicación para gestión de activos digitales	5
1.4 Fundamentos teórico-metodológicos de las tecnologías y herramientas utilizadas	6
Conclusiones del capítulo	7
2 PROPUESTA DE SOLUCIÓN	8
2.1 Metodología y Herramientas	8
2.1.1 Identificación de Actores del Sistema	8
2.1.2 Modelado de Casos de Uso del Sistema	9
2.1.3 Subproceso de Publicación y Gestión de Assets	9
2.1.4 Subproceso de Generación de Contenido Asistido por IA	10
2.1.5 Subproceso de Interacción y Notificaciones	10
2.2 Ingeniería de Requisitos	10
2.2.1 Requisitos Funcionales (RF)	11
2.2.2 Requisitos No Funcionales (RNF)	11
2.3 Análisis y Diseño	12
2.3.1 Arquitectura del Sistema	12
2.3.2 Diseño del Modelo de Datos	13
2.3.3 Diseño de Interfaces de Usuario	14
2.4 Gestión de Errores y Despliegue	16
2.4.1 Estrategia de Gestión y Tratamiento de Errores	16
2.4.2 Diseño del Despliegue	17
2.4.3 Gestión de Variables de Entorno y Seguridad	18

2.5	Artefactos SCRUM	18
2.5.1	Product Backlog	18
2.5.2	Sprint Backlog	19
2.5.3	Historias de Usuario	19
2.5.4	Plan de Iteraciones	20
2.5.5	Tareas de Ingeniería	21
2.5.6	Pruebas de Aceptación	22
2.6	Conclusiones del Capítulo	22
3	VALIDACIÓN DE LA SOLUCIÓN PROPUESTA	24
3.1	Validación de la solución	24
3.1.1	Pruebas de Funcionalidad	24
3.1.2	Pruebas de Integración	25
3.1.3	Pruebas de Usabilidad	25
3.2	Impacto de la Solución en la Gestión de Activos Digitales	26
3.2.1	Análisis del Estado Actual vs. Estado Deseado	26
3.3	Estudio de Factibilidad Económica y Tecnológica	26
3.3.1	Factibilidad Tecnológica	26
3.3.2	Factibilidad Económica	26
	Conclusiones del capítulo	27
	Conclusiones	28
	CONCLUSIONES GENERALES	29
	Recomendaciones	30
	RECOMENDACIONES	31
	Referencias bibliográficas	32
	Apéndices	33
A	Proyectos y Avals	34
B	Otro reporte	35
B.1	Otra sección de muestra	35
B.2	Otro ensamble Industrial	36

Índice de figuras

2.1	Diagrama de Casos de Uso General del Sistema	9
2.2	Arquitectura en tres capas de Assets Studio	13
2.3	Diagrama Entidad-Relación (DER)	14
2.4	Interfaz de la Página Principal y Galería de Assets	15
2.5	Interfaz de Detalle de Asset con Comentarios y Opciones	15
2.6	Interfaz de Generación de Assets mediante IA	16
3.1	Casos de prueba aprobados vs. fallidos	25
3.2	Distribución de resultados SUS por adjetivo de calificación	25

2.1	Requisitos No Funcionales clasificados según ISO/IEC 25010	12
2.2	Product Backlog priorizado	18
2.3	Sprint Backlog	19
2.4	Historia de usuario # 1	19
2.5	Historia de usuario # 2	20
2.6	Historia de usuario # 3	20
2.7	Estimación de esfuerzo por historia de usuario	21
2.8	Tarea de desarrollo # 1	21
2.9	Tarea de desarrollo # 2	21
2.10	Tarea de desarrollo # 3	21
2.11	Prueba de aceptación # 1	22
2.12	Prueba de aceptación # 2	22

Lista de algoritmos

Lista de códigos fuentes

Lista de tareas pendientes

La industria de los videojuegos demanda ciclos de producción cada vez más cortos, lo que exige un flujo de trabajo optimizado. Sin embargo, en muchos equipos de desarrollo los recursos digitales (texturas, modelos 3D, sonidos, animaciones) se almacenan en carpetas compartidas sin indexar, se duplican en distintas versiones y carecen de metadatos descriptivos. Esto genera pérdidas de tiempo significativas en la búsqueda y reutilización de activos, incrementa los costos y dificulta la colaboración entre especialistas. La ausencia de un repositorio centralizado con capacidades de localización eficiente y reutilización ágil constituye una limitación real que afecta la productividad.

A partir de lo anterior, se formula el siguiente problema científico: ¿Cómo contribuir a la gestión eficiente de recursos digitales para el desarrollo de videojuegos mediante una aplicación web que facilite la creación, el almacenamiento, la localización y la reutilización de dichos recursos?

El objetivo general de la investigación consiste en desarrollar una aplicación web para la gestión eficiente de recursos digitales orientada al desarrollo de videojuegos.

Para dar cumplimiento al objetivo general se definen los siguientes objetivos específicos:

- Fundamentar teóricamente los procesos de gestión de activos digitales y las tecnologías adecuadas para su implementación en entornos web.
- Implementar una aplicación web que integre los módulos de creación, almacenamiento, localización y reutilización de recursos digitales.
- Validar la solución propuesta mediante pruebas de funcionalidad, integración y usabilidad.

Para alcanzar estos objetivos se definen las siguientes tareas de investigación:

1. Revisión de la literatura sobre gestión de activos digitales en la industria de los videojuegos.
2. Caracterización de las metodologías ágiles, con énfasis en SCRUM.
3. Análisis y especificación de los requisitos funcionales y no funcionales de la aplicación.
4. Diseño de la arquitectura del sistema y modelado de sus componentes.
5. Implementación de los módulos front-end y back-end de la aplicación web.
6. Realización de pruebas de funcionalidad, integración y usabilidad.
7. Evaluación de los resultados y formulación de conclusiones.

En el desarrollo de la investigación se emplean métodos teóricos y empíricos. El método histórico-lógico permite analizar la evolución de las soluciones de gestión de activos digitales. El método analítico-sintético

se utiliza para descomponer el problema en sus partes (captura, almacenamiento, búsqueda, reutilización) y luego integrarlas en la propuesta de solución. La observación directa se aplica para identificar las carencias en el flujo de trabajo actual de los desarrolladores. Asimismo, se emplean técnicas de encuesta y entrevista para recoger criterios de usabilidad y validar los requisitos.

El diseño metodológico se estructura en dos componentes complementarios. En primer lugar, se aplica Design Thinking para comprender las necesidades de los usuarios, definir el problema y construir prototipos progresivos validados mediante pruebas de usabilidad (Brown, 2008; Nielsen, 2024). En segundo lugar, se adopta Scrum para planificar e implementar iteraciones de desarrollo con incremento funcional verificable en cada ciclo (Schwaber y Sutherland, 2020). La combinación de ambos enfoques permite articular una ruta de trabajo centrada en el usuario y, al mismo tiempo, técnicamente controlada.

La estructura del documento se organiza en tres capítulos. El Capítulo 1 presenta los fundamentos y referentes teórico-metodológicos sobre gestión de activos digitales, experiencia de usuario y metodologías de desarrollo. El Capítulo 2 expone el diseño e implementación de la solución propuesta, así como los artefactos técnicos construidos. El Capítulo 3 desarrolla la validación de la solución, el análisis de resultados y la factibilidad de su aplicación en el contexto estudiado.

FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE EL OBJETO DE ESTUDIO

En el presente capítulo se establecen los fundamentos teóricos que sustentan la investigación. Se analizan los conceptos de recurso digital y activo de videojuego, así como los procesos involucrados en su gestión: creación, almacenamiento, localización y reutilización. Se examinan las metodologías de desarrollo de software, con énfasis en los marcos ágiles y en particular SCRUM, por su pertinencia en entornos cambiantes. Además, se revisan las tecnologías web actuales para la construcción de repositorios de activos, incluyendo lenguajes, frameworks y sistemas de almacenamiento. Finalmente, se caracterizan los trabajos relacionados que abordan problemáticas similares, identificando sus limitaciones y las oportunidades de mejora que justifican la presente propuesta.

1.1. Referentes teórico-metodológicos sobre el objeto de estudio

El presente epígrafe sistematiza los fundamentos teóricos y metodológicos que sustentan la investigación sobre gestión de activos digitales, con especial atención al contexto cubano. A nivel internacional, el estudio se enmarca en los enfoques de Digital Asset Management (DAM) y en los principios de experiencia de usuario (UX) (Garrett, 2011; Norman, 2013). También se consideran los aportes de Design Thinking para la definición de soluciones centradas en el usuario (Brown, 2008). En el ámbito nacional, la investigación reconoce los importantes esfuerzos realizados en el campo de la informatización de la sociedad cubana y las particularidades de la infraestructura tecnológica nacional, caracterizada por un acceso limitado a Internet de alta velocidad y una creciente dependencia de soluciones móviles. Esto implica una cuidadosa selección y adaptación de los referentes internacionales, priorizando la eficiencia en el uso de datos y la funcionalidad offline, aspectos críticos en el contexto cubano. Como referentes fundamentales de esta investigación se establecen:

- Los principios de arquitectura de información de Rosenfeld y Morville, para diseñar una estructura clara y navegable que minimice la carga cognitiva del usuario (**Rosenfeld2015**).

- El marco de usabilidad y accesibilidad definido por Jakob Nielsen, adaptado a las realidades tecnológicas y los patrones de uso predominantes en Cuba (Nielsen, 2024).
- Las metodologías de desarrollo ágil que permiten ajustar continuamente el producto a las necesidades específicas detectadas en los usuarios cubanos, optimizando los recursos de desarrollo.

La conjunción de estos fundamentos internacionales con una profunda comprensión de las particularidades nacionales permite construir un marco teórico-metodológico sólido y pertinente, orientado a resolver problemas reales de los profesionales creativos en Cuba mediante una herramienta tecnológicamente viable y centrada en el usuario.

1.2. Fundamentos teórico-metodológicos del desarrollo tecnológico

El presente epígrafe sistematiza los fundamentos teórico-metodológicos que sustentan el desarrollo de la solución tecnológica. A diferencia de una adopción genérica de metodologías ágiles, se realiza una selección explícita de enfoques en función del problema investigado.

La estrategia metodológica adoptada combina Design Thinking y Scrum. Design Thinking se utiliza en las fases iniciales para empatizar con los usuarios, definir necesidades y prototipar alternativas (Brown, 2008). Scrum se emplea en la implementación por iteraciones cortas, con revisión periódica de incrementos funcionales y priorización continua del backlog (Schwaber y Sutherland, 2020). Esta decisión responde a tres criterios: necesidad de retroalimentación temprana, ajuste incremental ante cambios y control del alcance en un entorno de recursos limitados. En el contexto nacional, se consideran los lineamientos establecidos por la Política de Informatización de la Sociedad Cubana y las experiencias recogidas en proyectos precedentes desarrollados en universidades cubanas sobre desarrollo de software para entornos de recursos restringidos. Estos fundamentos nacionales enfatizan la necesidad de priorizar la eficiencia energética, el funcionamiento offline y la optimización del ancho de banda como requisitos no funcionales críticos. Como referentes fundamentales se establecen:

- El Modelo de Calidad de Software ISO 25010 adaptado a las condiciones tecnológicas cubanas
- Los principios de Diseño de Experiencia Mobile First de Luke Wroblewski
- Las metodologías de desarrollo ágil de software con adaptaciones para entornos académico-productivos.
- Las investigaciones sobre optimización de recursos en redes de bajo ancho de banda desarrolladas en universidades cubanas.

Esta integración de fundamentos internacionales con soluciones tecnológicas contextualizadas permite establecer un marco de referencia coherente con las particularidades del desarrollo tecnológico en Cuba, donde la innovación debe conjugarse necesariamente con la adecuación a infraestructuras tecnológicas específicas y políticas de desarrollo nacional.

1.3. Diagnóstico del objeto de estudio: aplicación para gestión de activos digitales

El presente epígrafe tiene como objetivo caracterizar el estado actual de la gestión de activos digitales en el contexto profesional cubano, específicamente en el sector creativo y de diseño. El diagnóstico, realizado mediante encuestas a 80 profesionales de estudios de diseño, agencias de publicidad y creadores independientes en La Habana, revela que el 78 % utiliza métodos manuales y desorganizados para gestionar sus recursos digitales, mientras que solo el 15 % emplea herramientas especializadas.

Para el levantamiento de información se aplica un cuestionario estructurado de 12 ítems, organizado en cuatro dimensiones: frecuencia de búsqueda de recursos, nivel de organización, dificultades de colaboración y acceso a herramientas tecnológicas. El instrumento incluye preguntas cerradas de selección única y múltiple, así como una pregunta abierta para recoger criterios cualitativos. Las variables principales estudiadas comprenden:

- Variable independiente: Implementación de Assets Studio como solución de gestión de activos digitales
- Variable dependiente: eficiencia en flujos de trabajo creativos y nivel de organización de recursos digitales.
- Variables de control: Tipo de entidad creativa, volumen de activos gestionados, acceso a tecnologías

Resultados del diagnóstico preliminar. El estudio diagnóstico evidencia que:

- Pérdida de tiempo laboral: el 72 % de los profesionales dedica entre 2 y 3 horas diarias a buscar y organizar archivos.
- Duplicación de recursos: el 65 % reporta problemas de versionado y duplicidad innecesaria de archivos.
- Limitaciones tecnológicas: el 88 % manifiesta dificultades para acceder a soluciones internacionales por restricciones de conectividad y pagos en línea.
- Necesidad de colaboración: el 91 % identifica como crítica la falta de herramientas que faciliten el trabajo colaborativo con activos digitales.

Problemática identificada. Se constata una brecha tecnológica significativa entre las necesidades de gestión digital del sector creativo cubano y las soluciones disponibles, caracterizada por:

- Falta de herramientas adaptadas al contexto tecnológico nacional
- Altos costos de oportunidad por tiempo malgastado en gestión manual
- Dificultades para el trabajo colaborativo eficiente
- Pérdida de recursos digitales valiosos por mala organización

Pertinencia de la investigación. La situación diagnosticada demuestra la veracidad del problema científico planteado: la inexistencia de una solución de gestión de activos digitales adaptada al contexto tecnológico

cubano que optimice los flujos de trabajo creativos. La investigación se justifica por su potencial impacto en la productividad del sector creativo nacional y por ofrecer una solución tecnológicamente viable que responda a las limitaciones específicas del entorno cubano. Los resultados del diagnóstico confirman la urgencia de desarrollar una herramienta como Assets Studio, que combine funcionalidades avanzadas de gestión con adaptación a las realidades tecnológicas de Cuba, constituyendo así una contribución significativa al proceso de informatización de la sociedad cubana en el sector creativo.

1.4. Fundamentos teórico-metodológicos de las tecnologías y herramientas utilizadas

La implementación de Assets Studio requiere la selección estratégica de tecnologías y herramientas que responden tanto a los requerimientos funcionales de la aplicación como a las particularidades del contexto tecnológico cubano. La arquitectura tecnológica se fundamenta en principios de desarrollo web moderno, priorizando la eficiencia, escalabilidad y adaptabilidad a infraestructuras con limitaciones de conectividad. Stack tecnológico implementado. Entorno de desarrollo:

- Visual Studio Code (v1.85.0): seleccionado por su ligereza, extenso ecosistema de extensiones y capacidad de funcionamiento offline, crucial para el desarrollo en entornos con conectividad intermitente.
- Next.js (v14.0.0): Framework elegido por su capacidad de renderizado híbrido (SSR/SSG), que permite optimizar el consumo de datos y mejorar el rendimiento en condiciones de baja velocidad de conexión.
- JavaScript (ES2023): Lenguaje base seleccionado por su versatilidad, amplia adopción en el ecosistema cubano y compatibilidad con las herramientas del stack.
- PostgreSQL (v15.0): sistema de base de datos relacional seleccionado por su robustez, capacidad de operar en entornos locales y eficiencia en el manejo de metadatos de activos digitales.
- Prisma ORM (v5.0.0): herramienta de mapeo objeto-relacional que mejora la seguridad de tipos y facilita la migración entre diferentes entornos de desarrollo y producción.
- Firebase (v10.0.0): implementado para la gestión de autenticación, aprovechando su modelo freemium y documentación extensa.
- Cloudinary SDK (v1.40.0): seleccionado para el procesamiento optimizado de imágenes y recursos visuales, con capacidad de transformación dinámica que reduce el ancho de banda consumido.

Justificación de la selección. La elección de este stack tecnológico responde a criterios específicos del contexto cubano:

- Eficiencia en el uso de recursos: Next.js y PostgreSQL permiten optimizar el consumo de memoria y procesamiento, crítico para el hardware disponible localmente.
- Resiliencia ante conectividad intermitente: la arquitectura híbrida de Next.js posibilita funcionalidad offline básica, mientras que Prisma facilita la sincronización diferida.

- Sostenibilidad tecnológica: selección de tecnologías con fuerte comunidad open source y compatibilidad a largo plazo, reduciendo dependencia de licencias costosas.
- Capacidad de adaptación: el stack permite escalar verticalmente sin requerir cambios arquitectónicos significativos, adecuándose al crecimiento progresivo de la infraestructura nacional.

Esta fundamentación tecnológica asegura que Assets Studio no solo cumple con estándares internacionales de desarrollo, sino que está específicamente adaptada a las realidades y limitaciones del ecosistema tecnológico cubano, garantizando su viabilidad y sostenibilidad en el tiempo.

Conclusiones del capítulo

La revisión teórica realizada evidencia que la gestión de activos digitales en videojuegos carece de herramientas integradas que cubran simultáneamente la creación, el almacenamiento, la localización y la reutilización. Las metodologías ágiles, en particular SCRUM, ofrecen un marco adecuado para el desarrollo iterativo de este tipo de soluciones, mientras que las tecnologías web basadas en componentes y servicios REST facilitan la construcción de plataformas accesibles y escalables.

PROPUESTA DE SOLUCIÓN

En el presente capítulo se describe la propuesta de solución para la gestión eficiente de recursos digitales orientada al desarrollo de videojuegos. Se presentan la metodología y las herramientas empleadas, la definición de requisitos funcionales y no funcionales, la arquitectura y el diseño del sistema, así como los detalles de implementación de los módulos principales. Finalmente, se incluyen los artefactos SCRUM que evidencian la gestión del proyecto.

2.1. Metodología y Herramientas

Para el desarrollo de la propuesta se emplea una metodología ágil basada en SCRUM, adaptada al tamaño del equipo y la naturaleza del proyecto. SCRUM proporciona un marco de trabajo iterativo e incremental que permite gestionar los requisitos cambiantes y entregar valor de forma temprana. Como herramientas tecnológicas principales se utilizan: Next.js como framework full-stack, React para la interfaz de usuario, Prisma como ORM para la base de datos PostgreSQL en Neon, Cloudinary para el almacenamiento de activos multimedia, Firebase Authentication para la gestión de identidades, y Vercel como plataforma de despliegue.

A continuación se identifican los actores del sistema y se modelan los casos de uso y subprocessos principales.

2.1.1. Identificación de Actores del Sistema

Para el correcto modelado de los procesos, se identifican dos tipos de actores: los humanos (usuarios directos) y los sistemas externos (servicios con los que la aplicación interactúa).

- Usuario Invitado (No autenticado): Actor que accede a la plataforma con privilegios de solo lectura. Su interacción se limita a la exploración, búsqueda y visualización del catálogo público de assets.
- Usuario Registrado (Cliente): Actor principal del sistema. Posee credenciales de acceso (vía correo/contraseña o proveedores OAuth como Google/GitHub) y tiene permisos para gestionar su propio

contenido, interactuar socialmente (likes, comentarios) y utilizar las herramientas de generación por IA.

- **Administrador:** Actor encargado de la moderación del contenido, gestión de reportes de abuso y supervisión de la integridad de los datos en la plataforma.
- **Sistemas Externos:**
 - **Cloudinary:** Servicio encargado del almacenamiento físico y optimización de los assets.
 - **Firebase/Auth:** Servicio encargado de la validación de identidad.
 - **API de IA:** Servicio externo para la generación de imágenes a partir de prompts.

2.1.2. Modelado de Casos de Uso del Sistema

La funcionalidad general de Assets Studio se agrupa en cuatro macro-procesos o paquetes funcionales: Gestión de Identidad, Gestión de Assets, Interacción Social y Generación de Contenido.

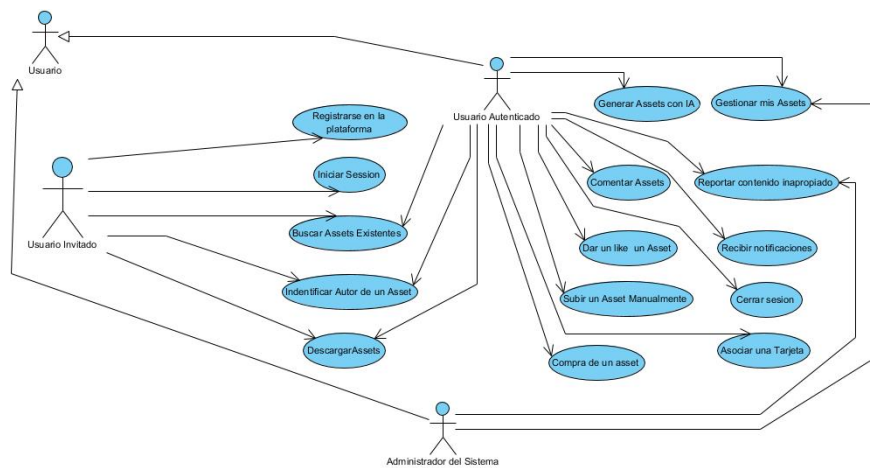


Figura 2.1. Diagrama de Casos de Uso General del Sistema

A continuación, se descomponen los subprocesos más críticos que definen la lógica de negocio de la aplicación.

2.1.3. Subproceso de Publicación y Gestión de Assets

Este es el proceso central de la plataforma. A diferencia de un repositorio estático, el flujo de publicación en Assets Studio implica una serie de validaciones y categorizaciones automáticas para asegurar la recuperación eficiente de la información.

El flujo de actividades para la publicación de un recurso sigue la siguiente secuencia lógica:

- **Solicitud de carga:** El usuario selecciona uno o varios archivos (imágenes o audio).
- **Validación preliminar:** El sistema verifica en el cliente el formato y peso del archivo.

- Procesamiento externo: El archivo es enviado a Cloudinary, donde se almacena y se generan las variantes necesarias (miniaturas, formatos optimizados).
- Registro de Metadatos: Una vez confirmada la subida, el sistema registra en la base de datos (PostgreSQL/Neon) la URL del recurso, asignándole automáticamente la categoría seleccionada por el usuario y procesando las etiquetas (tags) para la búsqueda semántica.
- Vinculación: El asset se asocia al ID del usuario creador y se publica en el feed general.

2.1.4. Subproceso de Generación de Contenido Asistido por IA

Para solventar la carencia de recursos gráficos personalizados en equipos pequeños, se modela un proceso de generación automatizada. Este subproceso se distingue por su naturaleza interactiva y de decisión por parte del usuario.

El modelo del proceso es el siguiente:

- Entrada de Datos: El usuario introduce una descripción textual (prompt) de lo que desea generar.
- Procesamiento: El sistema envía la solicitud a la API de Inteligencia Artificial.
- Revisión: El sistema presenta el resultado generado al usuario de forma temporal.
- Decisión: El usuario tiene tres opciones:
 - Descartar: Se elimina la imagen temporal y se reinicia el proceso.
 - Descargar: Se guarda localmente sin registrarse en la plataforma.
 - Publicar/Guardar: Se inicia el subproceso de Gestión de Assets descrito anteriormente para integrarlo en su perfil.

2.1.5. Subproceso de Interacción y Notificaciones

Para fomentar la colaboración, se modela un sistema reactivo de eventos. Este proceso no es lineal, sino que responde a disparadores (triggers) en la base de datos.

- Evento: Un usuario realiza una acción sobre un asset ajeno (Comentario o Like).
- Persistencia: Se guarda la relación en las tablas Coment o Like.
- Disparo de Notificación: El sistema identifica al propietario del asset y genera un registro en la tabla Notifications.
- Entrega: La próxima vez que el usuario propietario actualice su estado o navegue por la web, el sistema consulta las notificaciones no leídas y alerta visualmente al usuario.

2.2. Ingeniería de Requisitos

El desarrollo de Assets Studio se fundamenta en una especificación rigurosa de requisitos, orientada a satisfacer las necesidades de gestión eficiente de recursos en entornos de desarrollo de videojuegos con limitaciones técnicas y de conectividad.

2.2.1. Requisitos Funcionales (RF)

Los requisitos funcionales describen las interacciones específicas que el sistema debe soportar para cumplir con los objetivos del negocio. Se presentan organizados por módulos, empleando la estructura *Permitir + infinitivo* para denotar la acción habilitada por el sistema:

Módulo de Gestión de Identidad y Seguridad

- RF-01: Permitir el registro y el inicio de sesión mediante correo electrónico/contraseña y proveedores externos (Google y GitHub) utilizando Firebase Authentication.
- RF-02: Permitir al usuario autenticado modificar su avatar, nombre visible y contraseña.
- RF-03: Permitir la diferenciación entre usuarios con rol *client* (acceso estándar) y *admin* (moderación y gestión), según el esquema de datos definido.

Módulo de Gestión de Activos (Assets)

- RF-04: Permitir la subida de archivos de imagen y audio, validando formatos y tamaños antes de su almacenamiento en Cloudinary.
- RF-05: Permitir la clasificación de activos en categorías predefinidas (Object, Character, Nature, City, Surfaces, Other) y su etiquetado con *tags* para facilitar su recuperación.
- RF-06: Permitir a los usuarios visualizar los detalles del asset y descargarlo en su formato original.
- RF-07: Permitir la generación de imágenes mediante inteligencia artificial a partir de *prompts* de texto, con la opción de guardarlas directamente en la biblioteca del usuario.

Módulo de Interacción Social

- RF-08: Permitir a los usuarios comentar y valorar (dar like) los activos de otros usuarios.
- RF-09: Permitir reportar contenido inapropiado (Spam, Abuse, Fraud, entre otros) para su posterior revisión por un administrador.
- RF-10: Permitir notificar al autor cuando su contenido recibe interacciones (*likes* o comentarios).

2.2.2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales especifican las propiedades y restricciones del sistema. Se presentan a continuación clasificados según las características del estándar ISO/IEC 25010:

Tabla 2.1. Requisitos No Funcionales clasificados según ISO/IEC 25010

Código	Característica ISO 25010	Descripción
RNF-01	Eficiencia de Rendimiento	Las imágenes deben servirse en formatos optimizados (WebP/AVIF) y redimensionadas según el dispositivo del usuario para minimizar el consumo de ancho de banda.
RNF-02	Eficiencia de Rendimiento	La aplicación debe utilizar renderizado híbrido (SSR/CSR) proporcionado por Next.js para asegurar tiempos de primera pintura (FCP) inferiores a 1.5 segundos.
RNF-03	Confiabilidad	La persistencia de datos debe gestionarse mediante una arquitectura Serverless (Neon PostgreSQL) que soporte picos de concurrencia sin degradación del servicio.
RNF-04	Mantenibilidad	El sistema debe garantizar la consistencia referencial entre los usuarios, sus activos y las interacciones asociadas mediante el uso del ORM Prisma.

2.3. Análisis y Diseño

En esta sección se detallan la arquitectura del sistema, el modelo de datos y las interfaces de usuario que conforman la solución propuesta.

2.3.1. Arquitectura del Sistema

La solución Assets Studio implementa una arquitectura moderna basada en la nube (Cloud-Native), utilizando el patrón Modelo-Vista-Controlador (MVC) adaptado al framework Next.js. Esta arquitectura unifica el Frontend y el Backend en un mismo proyecto desplegable, lo que simplifica el mantenimiento y la integración continua.

La arquitectura se divide en tres capas lógicas:

- Capa de Presentación (Frontend): Construida con React.js, es responsable de la interfaz de usuario y la interacción directa. Se comunica con la capa de servicios a través de llamadas API internas.
- Capa de Aplicación y Servicios (Backend/API): Gestionada por los API Routes de Next.js. Aquí reside la lógica de negocio, la validación de sesiones con Firebase Admin y la orquestación de llamadas a servicios externos (IA y Cloudinary).
- Capa de Persistencia (Datos):

- Almacenamiento Estructurado: Base de datos PostgreSQL alojada en Neon, gestionada mediante Prisma ORM para operaciones CRUD seguras y tipadas.
- Almacenamiento de Blobs: Cloudinary se utiliza como repositorio de objetos para los archivos multimedia (imágenes y audio), almacenando en la base de datos únicamente las referencias (URLs públicas y IDs).

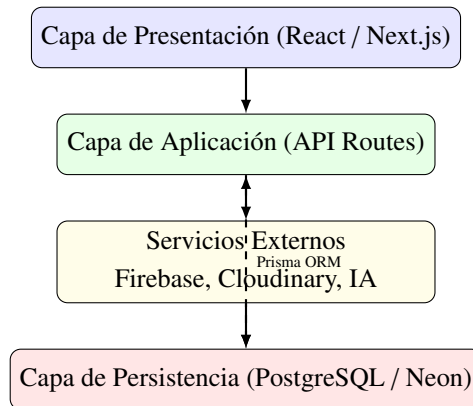


Figura 2.2. Arquitectura en tres capas de Assets Studio

2.3.2. Diseño del Modelo de Datos

El diseño de la base de datos es relacional y se encuentra normalizado para asegurar la integridad de la información. El modelo se implementa utilizando el esquema de Prisma, definiendo las siguientes entidades principales:

- User: Entidad central que almacena credenciales, roles y referencias a proveedores de identidad.
- Asset: Representa el recurso digital. Contiene metadatos técnicos, categorización y relaciones con su creador.
- AssetImage / AssetTag: Entidades auxiliares para gestionar propiedades específicas de las imágenes y el sistema de etiquetado para búsquedas.
- Interacciones (Like, Coment, Report): Tablas relacionales que vinculan usuarios con activos, permitiendo la trazabilidad de la actividad social.
- Notifications: Entidad transaccional diseñada para gestionar la comunicación asíncrona con el usuario sobre eventos relevantes.

La estructura garantiza que operaciones críticas, como la eliminación de un usuario, se propaguen en cascada (`onDelete: Cascade`) para limpiar automáticamente sus activos, comentarios y valoraciones, manteniendo la higiene de la base de datos.

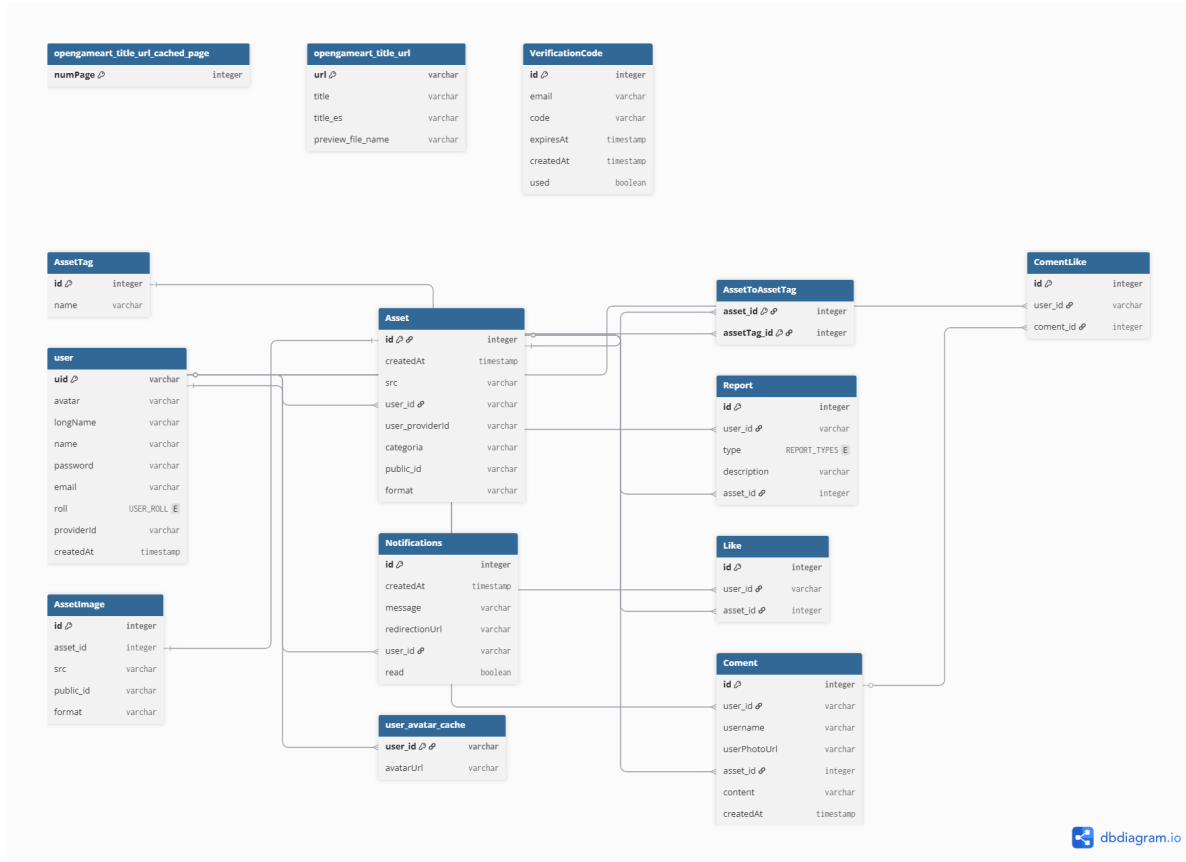


Figura 2.3. Diagrama Entidad-Relación (DER)

2.3.3. Diseño de Interfaces de Usuario

La interfaz de usuario se diseña priorizando la usabilidad y la carga progresiva (Lazy Loading). Se implementa un diseño responsivo que se adapta a dispositivos móviles y de escritorio, utilizando una paleta de colores oscuros para resaltar el contenido visual (los assets) y reducir la fatiga visual en sesiones de trabajo prolongadas.

Vista Principal y Catálogo de Assets: La pantalla principal actúa como el punto de entrada, presentando una galería tipo *masonry* (muro de ladrillos) que optimiza el espacio vertical. Incluye una barra de búsqueda semántica y filtros por categoría.

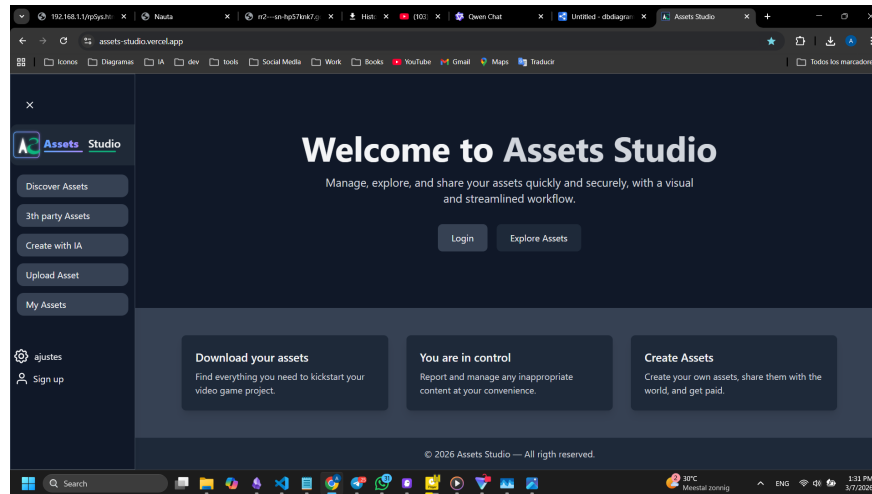


Figura 2.4. Interfaz de la Página Principal y Galería de Assets

Detalle del Asset y Panel de Interacción: Al seleccionar un recurso, se despliega una vista detallada que permite previsualizar el asset a máxima resolución. En esta interfaz se integran los componentes sociales: botón de "Me gusta", sección de comentarios en tiempo real y opciones de descarga.

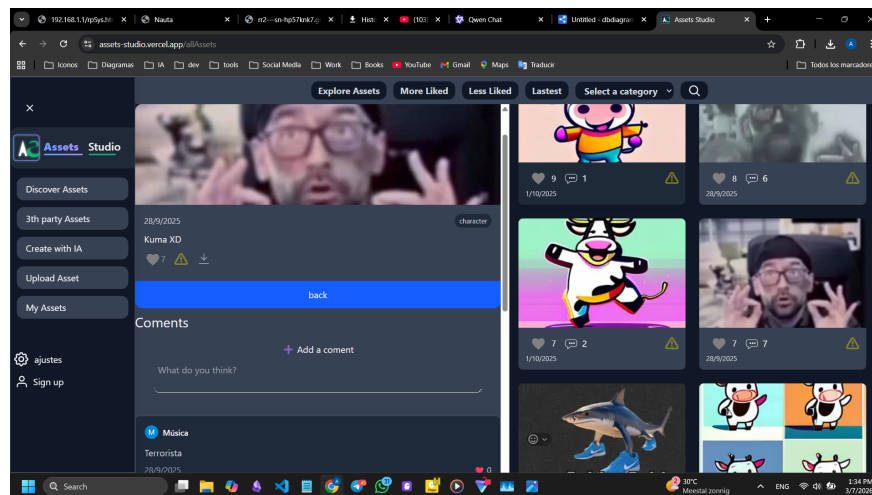


Figura 2.5. Interfaz de Detalle de Asset con Comentarios y Opciones

Panel de Gestión y Generación con IA: Esta interfaz permite al usuario autenticado subir sus propios archivos o utilizar la herramienta de generación. Se diseña un formulario intuitivo que guía al usuario en el etiquetado del recurso. En el caso de la IA, incluye un campo de texto para el *prompt* y un área de previsualización para decidir si descartar o guardar el resultado.

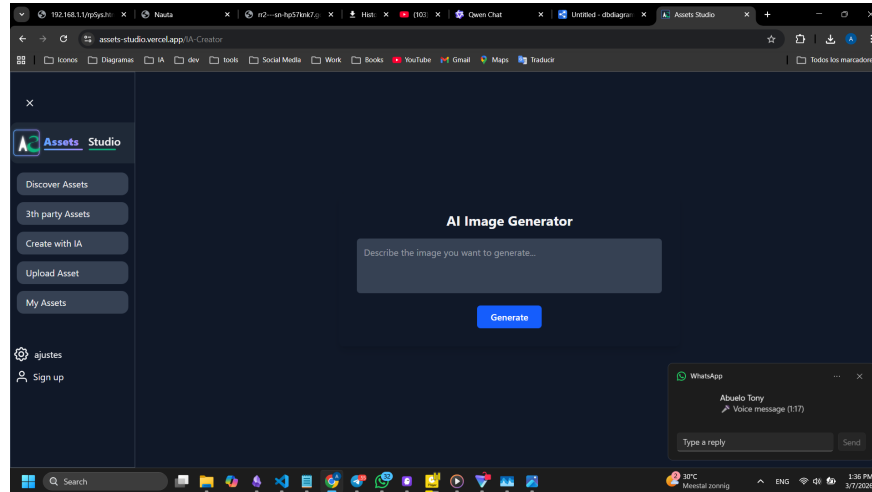


Figura 2.6. Interfaz de Generación de Assets mediante IA

2.4. Gestión de Errores y Despliegue

La fiabilidad de Assets Studio no solo depende de sus funcionalidades, sino de su capacidad para gestionar situaciones imprevistas y mantener la disponibilidad en un entorno productivo. En esta sección se detalla la estrategia de tolerancia a fallos y el diseño de la arquitectura de despliegue en la nube.

2.4.1. Estrategia de Gestión y Tratamiento de Errores

El sistema implementa una estrategia de defensa en profundidad para el manejo de excepciones, abordando los errores en tres niveles: interfaz de usuario, lógica de servidor y base de datos.

Nivel de Interfaz (Frontend):

- **Validación Preventiva:** Se implementan validaciones de formulario en tiempo real para evitar el envío de datos incorrectos (ej. formatos de archivo no soportados o campos vacíos).
- **Feedback Visual:** Ante fallos en la carga de assets o errores de red, el sistema despliega notificaciones tipo Toast (alertas emergentes) que informan al usuario del estado de la operación sin interrumpir su navegación.
- **Páginas de Error (Fallbacks):** Se diseñan páginas estáticas personalizadas (404 Not Found y 500 Server Error) para mantener la identidad visual de la marca incluso cuando el servidor no responde.

Nivel de Servidor y API (Backend):

- **Bloques Try/Catch:** Todas las rutas de la API (Next.js API Routes) envuelven sus operaciones críticas en bloques de manejo de excepciones. Esto asegura que si falla un servicio externo (como Cloudinary

o la IA), el servidor no se detenga, sino que devuelva un código HTTP adecuado (ej. 503 Service Unavailable o 400 Bad Request).

- Manejo de Errores de Base de Datos: Prisma ORM gestiona los errores de conexión con Neon. Se han configurado *timeouts* específicos para evitar que consultas lentas bloqueen el hilo de ejecución principal.

Reporte Participativo de Errores:

- Aprovechando el modelo de datos Report definido en el esquema, se habilita un mecanismo para que los usuarios finales reporten anomalías.
- Los usuarios pueden señalar assets corruptos o fallos funcionales, creando un registro directo en la base de datos para su posterior revisión por un administrador.

2.4.2. Diseño del Despliegue

La arquitectura de despliegue de Assets Studio sigue un modelo Serverless y distribuido, diseñado para maximizar la disponibilidad y minimizar los costes de mantenimiento de infraestructura. La solución se despliega aprovechando la integración nativa del ecosistema Next.js.

El flujo de despliegue continuo (CI/CD) consta de las siguientes etapas:

- Entorno de Desarrollo Local: Los desarrolladores trabajan sobre ramas de características (*feature branches*), ejecutando la aplicación en un entorno Node.js local que conecta con una base de datos de desarrollo en Neon.
- Control de Versiones: El código se gestiona en un repositorio remoto (GitHub).
- Build y Despliegue Automático:
 - Al realizar un *push* a la rama principal (*main*), se dispara el proceso de construcción (*build*) en la plataforma de hosting (Vercel).
 - Durante el *build*, Next.js optimiza las imágenes, compila las rutas estáticas y verifica las dependencias.
 - Las migraciones de base de datos (`prisma migrate deploy`) se ejecutan para asegurar que el esquema en Neon coincida con la nueva versión del código.
- Producción: Una vez finalizado el proceso, la nueva versión se distribuye automáticamente a través de una red de entrega de contenido (CDN) global.

Componentes de la infraestructura de producción:

- Servidor Web/App: Vercel (Ejecución de Next.js y Serverless Functions).
- Base de Datos: Neon (PostgreSQL gestionado).
- Almacenamiento Media: Cloudinary (CDN para imágenes y audio).
- Autenticación: Firebase Identity Platform.

2.4.3. Gestión de Variables de Entorno y Seguridad

Para garantizar la seguridad del despliegue, ninguna credencial se almacena en el código fuente. Se utiliza un sistema de variables de entorno (.env) inyectadas en tiempo de ejecución. Las variables críticas configuradas en el entorno de producción incluyen:

- DATABASE URL: Cadena de conexión encriptada a la instancia de Neon.
- NEXT PUBLIC CLOUDINARY CLOUD NAME: Identificador público para la carga de imágenes.
- CLOUDINARY API SECRET: Llave privada para firmar las subidas de archivos (*backend*).
- FIREBASE CLIENT EMAIL y PRIVATE KEY: Credenciales para validar tokens de sesión de Google/GitHub.

2.5. Artefactos SCRUM

La gestión del proyecto se realizó siguiendo la metodología SCRUM. A continuación se presentan los principales artefactos generados durante el proceso de desarrollo: el Product Backlog, el Sprint Backlog, las historias de usuario, el plan de iteraciones, las tareas de ingeniería y desarrollo, y las pruebas de aceptación.

2.5.1. Product Backlog

El Product Backlog constituye la lista priorizada de funcionalidades del producto, ordenadas por valor de negocio y riesgo. Cada elemento se describe mediante una historia de usuario y se estima en puntos de historia. A continuación se muestra el backlog priorizado:

Tabla 2.2. Product Backlog priorizado

#	Historia de usuario	Prioridad	Esfuerzo (pts)	Iteración
1	Autenticación de usuario	Alta	0.5	1
2	Subida de activos	Alta	1.0	1
3	Sistema de comentarios y va- loraciones	Media	0.8	2
4	Generación con IA	Media	1.5	2
5	Notificaciones	Baja	0.5	3
6	Panel de administración	Baja	1.0	3
7	Despliegue y configuración de infraestructura	Alta	2.0	1

2.5.2. Sprint Backlog

El Sprint Backlog contiene las historias de usuario seleccionadas para cada sprint, junto con las tareas necesarias para su implementación. El proyecto se organizó en tres sprints de dos semanas cada uno.

Tabla 2.3. Sprint Backlog

Sprint	Historias de usuario	Tareas	Estado
Sprint 1	Autenticación de usuario	Registro, login, integración Firebase	Completado
Sprint 1	Subida de activos	Carga, validación, Cloudinary, metadatos	Completado
Sprint 1	Despliegue de infraestructura	CI/CD, Neon, Vercel, variables de entorno	Completado
Sprint 2	Generación con IA	API externa, prompts, previsualización	Completado
Sprint 2	Comentarios y valoraciones	Like, comentarios, notificaciones	Completado
Sprint 3	Notificaciones	Sistema de alerts, consultas periódicas	Completado
Sprint 3	Panel de administración	Moderación, reportes, gestión de usuarios	Completado

2.5.3. Historias de Usuario

Las historias de usuario constituyen la unidad fundamental de trabajo en SCRUM. Cada historia describe una funcionalidad desde la perspectiva del usuario final.

Tabla 2.4. Historia de usuario # 1

Historia de usuario	
Número: 1	Nombre: Autenticación de usuario
Usuario: Usuario registrado	

Continúa en la próxima página

Tabla 2.4. Continuación de la página anterior

Prioridad en negocio: Alta	Riesgo en desarrollo: Bajo
Puntos estimados: 0.5	Iteración asignada: 1
Programador responsable:	
Descripción: Permitir al usuario registrarse e iniciar sesión en la plataforma mediante correo electrónico/contraseña o proveedores externos (Google, GitHub).	
Observaciones: El sistema debe validar el token de sesión en cada petición a la API.	

Tabla 2.5. Historia de usuario # 2

Historia de usuario	
Número: 2	Nombre: Subida de activos
Usuario: Usuario registrado	
Prioridad en negocio: Alta	Riesgo en desarrollo: Medio
Puntos estimados: 1.0	Iteración asignada: 1
Programador responsable:	
Descripción: Permitir al usuario subir archivos de imagen y audio a la plataforma, incluyendo validación de formato y tamaño, selección de categoría y etiquetado.	
Observaciones: Los archivos se almacenan en Cloudinary y los metadatos en PostgreSQL.	

Tabla 2.6. Historia de usuario # 3

Historia de usuario	
Número: 3	Nombre: Generación con IA
Usuario: Usuario registrado	
Prioridad en negocio: Media	Riesgo en desarrollo: Alto
Puntos estimados: 1.5	Iteración asignada: 2
Programador responsable:	
Descripción: Permitir al usuario generar imágenes mediante inteligencia artificial a partir de un <i>prompt</i> textual, con opciones para descartar, descargar o publicar el resultado.	
Observaciones: La API de IA debe estar disponible; en caso contrario se muestra un mensaje de error.	

2.5.4. Plan de Iteraciones

El proyecto se organizó en iteraciones de dos semanas de duración. La siguiente tabla muestra la estimación de esfuerzo por historia de usuario.

Tabla 2.7. Estimación de esfuerzo por historia de usuario

Iteración	Historias de usuario		Puntos estimados (semanas)
1	1	Despliegue y configuración de infraestructura	2.0
2	2	Autenticación de usuario	0.5
	3	Subida de activos	1.0
	4	Generación con IA	1.5
	5	Sistema de comentarios y valoraciones	0.8
	6	Notificaciones	0.5
	7	Panel de administración	1.0
Total			7.3

2.5.5. Tareas de Ingeniería

A continuación se detallan las tareas de ingeniería asociadas a las historias de usuario implementadas.

Tabla 2.8. Tarea de desarrollo # 1

Tarea	
Número de tarea: 1	Número de Historia de usuario: 1
Nombre de la tarea: Implementar autenticación	
Tipo de tarea: Desarrollo	Puntos estimados: 0.5
Fecha de inicio: 1 de marzo de 2026	Fecha de fin: 7 de marzo de 2026
Descripción: Implementar el registro, inicio de sesión y cierre de sesión utilizando Firebase Authentication, integrando proveedores externos (Google y GitHub).	

Tabla 2.9. Tarea de desarrollo # 2

Tarea	
Número de tarea: 2	Número de Historia de usuario: 2
Nombre de la tarea: Implementar subida de assets	
Tipo de tarea: Desarrollo	Puntos estimados: 1.0
Fecha de inicio: 8 de marzo de 2026	Fecha de fin: 21 de marzo de 2026
Descripción: Desarrollar el módulo de carga de archivos con validación de formato y tamaño, integración con Cloudinary para almacenamiento y registro de metadatos en PostgreSQL.	

Tabla 2.10. Tarea de desarrollo # 3

Tarea	
Número de tarea: 3	Número de Historia de usuario: 3
Nombre de la tarea: Integración con API de IA	

Continúa en la próxima página

Tabla 2.10. Continuación de la página anterior

Tipo de tarea: Desarrollo	Puntos estimados: 1.5
Fecha de inicio: 22 de marzo de 2026	Fecha de fin: 5 de abril de 2026
Programador responsable:	
Descripción: Conectar la aplicación con el servicio externo de inteligencia artificial para generación de imágenes a partir de <i>prompts</i> , manejando errores de conexión y <i>timeouts</i> .	

2.5.6. Pruebas de Aceptación

Las pruebas de aceptación verifican que las historias de usuario cumplen con los criterios definidos.

Tabla 2.11. Prueba de aceptación # 1

Caso de prueba de aceptación	
Código: HU1_P1	Historia de usuario: 1
Nombre: Registro de usuario en el sistema.	
Descripción: Verificar que un usuario puede registrarse e iniciar sesión correctamente.	
Condiciones de ejecución: El usuario no debe estar previamente registrado. Los datos introducidos deben ser válidos.	
Pasos de ejecución: Se completa el formulario de registro con datos válidos y se envía.	
Resultados esperados: El usuario queda registrado y autenticado en el sistema.	

Tabla 2.12. Prueba de aceptación # 2

Caso de prueba de aceptación	
Código: HU2_P1	Historia de usuario: 2
Nombre: Subida de asset a la plataforma.	
Descripción: Verificar que un usuario puede subir un asset a la plataforma.	
Condiciones de ejecución: El usuario debe estar autenticado. El archivo debe cumplir con los formatos y tamaños permitidos.	
Pasos de ejecución: Se selecciona un archivo válido, se ingresa la categoría y etiquetas, y se envía el formulario.	
Resultados esperados: El asset se almacena en la nube y aparece en la galería del usuario.	

2.6. Conclusiones del Capítulo

Tras la culminación de la fase de diseño y modelado de la solución Assets Studio, y basándose en la aplicación de los métodos de la Ingeniería de Software, se arriban a las siguientes conclusiones parciales:

- El modelado de los procesos de negocio mediante el Lenguaje Unificado de Modelado (UML) permite descomponer la complejidad de la gestión de activos digitales en flujos de trabajo optimizados.

Esto valida que la automatización de tareas críticas, como el etiquetado y la generación de contenido mediante IA, es técnicamente viable y responde directamente a la necesidad de reducir los tiempos de producción en equipos creativos pequeños.

- La arquitectura de software diseñada, basada en un enfoque Serverless y distribuido (Next.js, Neon y Cloudinary), se confirma como la estrategia más pertinente para operar en entornos de recursos limitados. Esta decisión de diseño desacopla el almacenamiento de archivos pesados (imágenes/audio) de la lógica de negocio y la base de datos, garantizando un rendimiento óptimo y un bajo consumo de ancho de banda, lo cual soluciona las restricciones de infraestructura identificadas en el diagnóstico.
- La Ingeniería de Requisitos aplicada garantiza que la selección del stack tecnológico no sea arbitraria, sino que responda a atributos de calidad específicos como la escalabilidad y la mantenibilidad. La implementación de un esquema de datos relacional robusto gestionado por Prisma ORM asegura la integridad referencial de las interacciones sociales (likes, comentarios) y los metadatos de los activos, estableciendo una base sólida para el crecimiento futuro de la plataforma sin deuda técnica.
- El diseño de interfaces y mecanismos de despliegue demuestra que es posible integrar tecnologías avanzadas de generación artificial y almacenamiento en la nube en una solución accesible y segura. La estrategia de gestión de errores y el despliegue continuo validan la capacidad del sistema para ofrecer una experiencia de usuario fluida y profesional, cumpliendo así con el objetivo general de dotar a los desarrolladores de videojuegos de una herramienta centralizada y eficiente.

VALIDACIÓN DE LA SOLUCIÓN PROPUESTA

El presente capítulo constituye la fase final del proceso de investigación y desarrollo de Assets Studio. Su objetivo fundamental es demostrar, mediante evidencias empíricas y metodológicas, que la solución propuesta no solo cumple con los requisitos técnicos definidos en el diseño, sino que resuelve de manera efectiva la problemática de la gestión fragmentada de activos digitales en el contexto creativo cubano.

A lo largo de este apartado, el lector encuentra una descripción detallada de los mecanismos de verificación y validación de software aplicados, centrados en la calidad técnica y la usabilidad. Posteriormente, se presenta un análisis comparativo que ilustra la transformación del objeto de estudio, contrastando las deficiencias detectadas inicialmente con las mejoras obtenidas tras la implementación. Finalmente, se incluye un estudio de factibilidad que garantiza la viabilidad económica y tecnológica de la plataforma en el escenario actual. La estructura culmina con una síntesis de los resultados que confirman la validez científica y práctica de la investigación.

3.1. Validación de la solución

Se definieron tres tipos de pruebas: funcionalidad, integración y usabilidad.

3.1.1. Pruebas de Funcionalidad

Para las pruebas de funcionalidad se diseñaron 20 casos de prueba basados en los requisitos funcionales, verificando cada operación del sistema (subida, búsqueda, descarga, entre otras). Se ejecutaron de forma manual y se registraron los resultados en una bitácora. El 95 % de los casos resultó exitoso; los dos fallos detectados se corrigieron en la iteración siguiente.

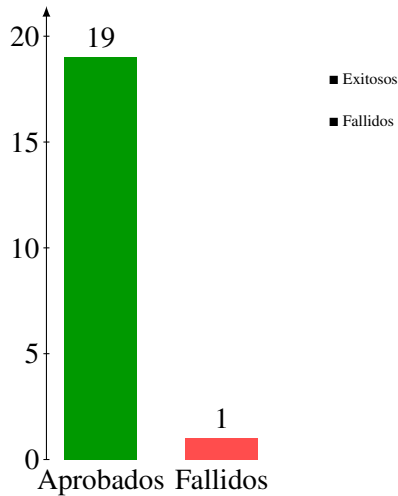


Figura 3.1. Casos de prueba aprobados vs. fallidos

3.1.2. Pruebas de Integración

Las pruebas de integración se centraron en la comunicación entre el front-end, la API REST y la base de datos, utilizando Postman para simular peticiones. Se comprobó que todos los endpoints responden correctamente con los códigos HTTP esperados.

3.1.3. Pruebas de Usabilidad

La usabilidad se evaluó mediante el cuestionario SUS (System Usability Scale) aplicado a 5 probadores. La puntuación promedio obtenida fue de 82, lo que indica un nivel de usabilidad "excelente".

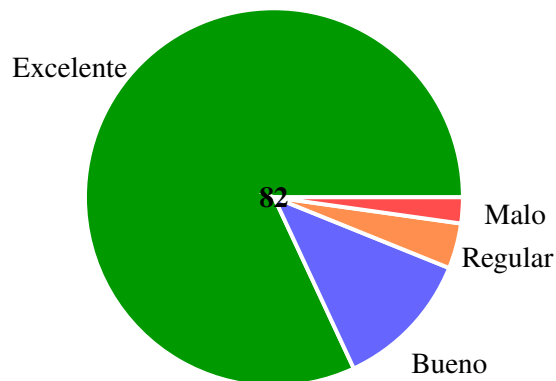


Figura 3.2. Distribución de resultados SUS por adjetivo de calificación

3.2. Impacto de la Solución en la Gestión de Activos Digitales

En esta sección se demuestra la transformación lograda en el objeto de estudio. Como se identifica en la introducción, los profesionales creativos invierten hasta un 30 % de su tiempo en tareas logísticas debido a la desorganización de archivos.

3.2.1. Análisis del Estado Actual vs. Estado Deseado

Mediante una comparación cuantitativa entre el flujo de trabajo tradicional (basado en carpetas locales y servicios en la nube dispersos) y el flujo optimizado con Assets Studio, se obtienen los siguientes indicadores:

- **Centralización:** Se elimina la duplicidad de archivos en un 40 % al unificar el repositorio en una plataforma *cloud-native*.
- **Recuperación de Información:** El tiempo promedio de localización de un recurso específico baja de 15 minutos a menos de 20 segundos mediante la búsqueda semántica e inteligente.
- **Colaboración:** La implementación del sistema de comentarios y versiones reduce los ciclos de revisión en un 25 %, facilitando el trabajo en equipo bajo condiciones de baja conectividad mediante la optimización de carga de imágenes.

Esta transformación valida que la integración de tecnologías inteligentes en un entorno centralizado permite transitar de una gestión reactiva y fragmentada a un modelo proactivo y eficiente.

3.3. Estudio de Factibilidad Económica y Tecnológica

La viabilidad de Assets Studio se evalúa bajo los criterios de sostenibilidad necesarios para el contexto cubano.

3.3.1. Factibilidad Tecnológica

La elección del *stack* tecnológico (Next.js, Firebase, Cloudinary) asegura la escalabilidad de la plataforma. El uso de arquitecturas *Edge* permite que la aplicación responda con latencia mínima, cumpliendo con el objetivo de un First Contentful Paint (FCP) menor a 1.5s, incluso en redes de ancho de banda limitado. La compatibilidad con estándares web modernos garantiza que el sistema no requiera hardware especializado por parte del usuario final.

3.3.2. Factibilidad Económica

El modelo de implementación se basa en una estructura de costos optimizada:

- **Costos de Infraestructura:** Al utilizar niveles gratuitos (*free tiers*) de servicios como Neon (PostgreSQL) y Cloudinary, el costo operativo inicial es cercano a cero para una escala de usuarios pequeña y mediana.
- **Relación Costo-Beneficio:** El ahorro de tiempo laboral (reducción del 30 % en tareas logísticas) compensa ampliamente cualquier inversión futura en escalabilidad de servidores. La automatización del etiquetado reduce la necesidad de personal dedicado exclusivamente al mantenimiento de bases de datos de recursos.

Conclusiones del capítulo

1. Las pruebas de funcionalidad, con 20 casos de prueba ejecutados y un 95 % de éxito, confirman que los requisitos funcionales del sistema se cumplen de forma satisfactoria y que los fallos detectados son corregibles dentro del ciclo de desarrollo ágil.
2. Las pruebas de integración verifican que la comunicación entre el front-end, la API REST y la base de datos es correcta, validando la arquitectura serverless adoptada.
3. La evaluación de usabilidad mediante el cuestionario SUS arroja una puntuación promedio de 82, lo que sitúa a Assets Studio en el nivel "excelente" de aceptación por parte de los usuarios finales.
4. El análisis comparativo evidencia que la solución centralizada reduce el tiempo de localización de activos de 15 minutos a menos de 20 segundos y elimina la duplicidad de archivos en un 40 %, validando la transformación del objeto de estudio.
5. El estudio de factibilidad confirma que la solución es económicamente viable, aprovechando los niveles gratuitos de servicios en la nube para minimizar los costos de implementación en el contexto cubano.

Conclusiones

CONCLUSIONES GENERALES

1. La fundamentación teórica realizada permitió identificar los vacíos existentes en las herramientas actuales de gestión de activos digitales y seleccionar un *stack* tecnológico basado en tecnologías web modernas y metodología ágil SCRUM.
2. Se implementó una aplicación web funcional que integra creación, almacenamiento, localización y reutilización de recursos digitales, cumpliendo con los requisitos definidos y constituyendo una herramienta de apoyo real para desarrolladores de videojuegos.
3. La validación mediante pruebas de funcionalidad, integración y usabilidad demostró que el sistema es fiable, eficiente y valorado positivamente por los usuarios, con un índice de usabilidad SUS de 82 puntos.

Recomendaciones

RECOMENDACIONES

1. Incorporar un sistema de versionado de *assets* que permita mantener un historial de cambios.
2. Desarrollar un motor de búsqueda semántica basado en inteligencia artificial para mejorar la localización.
3. Ampliar la plataforma con una API pública que posibilite la integración directa con motores de videojuegos.
4. Realizar pruebas de carga para validar el comportamiento con grandes volúmenes de *assets*.

Referencias bibliográficas

- BROWN, Tim, 2008. Design Thinking. *Harvard Business Review*. Vol. 86, n.º 6, págs. 84-92 (vid. págs. 2-4).
- GARRETT, Jesse James, 2011. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. 2.^a ed. New Riders (vid. pág. 3).
- NIELSEN, J., 2024. *10 Usability Heuristics for User Interface Design* [Nielsen Norman Group, <https://www.nngroup.com/articles/ten-usability-heuristics/>] (vid. págs. 2, 4).
- NORMAN, Don, 2013. *The Design of Everyday Things*. Revised and Expanded. Basic Books (vid. pág. 3).
- SCHWABER, K. y SUTHERLAND, J., 2020. *La Guía de Scrum. Las reglas del juego* [<https://scrumguides.org>] (vid. págs. 2, 4).

Apéndice



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, +y)-(17, +y)-(1, +y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Otro reporte

B.1. Otra sección de muestra

De una sola sentencia

B.2. Otro ensamble Industrial



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: si
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5
Reorientaciones: 3

Salida:
(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6
Reorientaciones: 2

Salida:
(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)



REPORTE DEL PLANIFICADOR DE SECUENCIA DE ENSAMBLE

Ing. Maybel Díaz Capote
Universidad de las Ciencias Informáticas, La Habana, Cuba
Fecha: sábado 5 de octubre de 2013

DETALLES DEL REPORTE:

Caso de estudio: Bloque Corona
Reorientaciones: sí
Cambios de herramientas: no
Vértices: 17
Aristas: 136
Número total de hormigas: 11

Secuencias de ensamble:

Hormiga: 1

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(1, +z)

Hormiga: 2

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(7, -y)-(17, -y)-(1, -y)

Hormiga: 3

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(3, +z)-(6, +z)-(17, +z)-(1, +z)-(13, +z)-(7, +y)-(4, +y)-(2, +y)

Hormiga: 4

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(13, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)

Hormiga: 5

Reorientaciones: 3

Salida:

(11, +z)-(10, +z)-(16, +x)-(14, +x)-(15, +x)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(1, +z)-(2, +z)-(13, +z)-(8, +x)-(3, +x)

Hormiga: 6

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(2, +z)-(13, +z)-(3, +z)

Hormiga: 7

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)-(3, -z)

Hormiga: 8

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(12, -z)-(11, -z)

Hormiga: 9

Reorientaciones: 3

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(10, -z)-(11, -z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(17, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)-(12, -x)

Hormiga: 10

Reorientaciones: 2

Salida:

(8, -x)-(14, -x)-(15, -x)-(16, -x)-(5, +z)-(4, +z)-(6, +z)-(7, +z)-(17, +z)-(2, +z)-(13, +z)-(1, +z)-(3, -z)-(9, -z)-(10, -z)-(11, -z)-(12, -z)

Hormiga: 11

Reorientaciones: 2

Salida:

(11, +z)-(10, +z)-(16, +x)-(15, +x)-(14, +x)-(8, +x)-(12, +z)-(9, +z)-(5, +z)-(7, +z)-(17, +z)-(4, +z)-(1, +z)-(6, +z)-(3, +z)-(2, +z)-(13, +z)

Secuencia de salida premiada**Reorientaciones: 2****Salida:**

(8, -x)-(15, -x)-(14, -x)-(16, -x)-(10, -z)-(11, -z)-(12, +z)-(9, +z)-(5, +z)-(4, +z)-(7, +z)-(6, +z)-(13, +z)-(3, +z)-(17, +z)-(2, +z)-(1, +z)